

Multidimensional difference arrays

via Discrete Calculus

What is this talk about?

- A runtime optimization of L update queries on an array of size N :

$$\mathcal{O}(N \cdot L) \longrightarrow \mathcal{O}(N + L)$$

What is this talk about?

- A runtime optimization of L update queries on an array of size N :

$$\mathcal{O}(N \cdot L) \longrightarrow \mathcal{O}(N + L)$$

- Simple heuristic for 1D, 2D and 3D arrays

What is this talk about?

- A runtime optimization of L update queries on an array of size N :

$$\mathcal{O}(N \cdot L) \longrightarrow \mathcal{O}(N + L)$$

- Simple heuristic for 1D, 2D and 3D arrays
- For general dimension: *Discrete Calculus*

What is this talk about?

- A runtime optimization of L update queries on an array of size N :

$$\mathcal{O}(N \cdot L) \longrightarrow \mathcal{O}(N + L)$$

- Simple heuristic for 1D, 2D and 3D arrays
- For general dimension: *Discrete Calculus*
- Further optimizations

One-dimensional difference arrays

	0	1	2	3	4	5	6	7	8
A:	*	*	*	*	*	*	*	*	*

↓
Apply Q₁

	0	1	2	3	4	5	6	7	8
A:	*	+1	+1	+1	*	*	*	*	*

↓
Apply Q₂

	0	1	2	3	4	5	6	7	8
A:	*	*	+2	+2	*	*	*	*	*

↓
Apply Q₃

	0	1	2	3	4	5	6	7	8
A:	*	*	*	*	*	-1	-1	-1	-1

Consider a one-dimensional array A of size 8.

Apply the queries

- Q₁: Update elements at indices in {1, 2, 3} by +1
- Q₂: Update elements at indices in {2, 3} by +2
- Q₃: Update elements at indices in {5, 6, 7, 8} by -1

One-dimensional difference arrays

Use two column layout and describe informal naive algorithm so that the runtime analysis is clearer.

Runtime analysis:

- One-dimensional array of size N
- Q queries for each of which the update range is at most of size N

Time complexity: $\mathcal{O}(Q \cdot N)$

We can do MUCH better!

One-dimensional difference arrays

We could also apply the query

- Q₁: Update elements at indices in {1, 2, 3} by +1

differently:

1. Initialize an array d of size 8
2. Add +1 at index 1
3. Add -1 at index 4
4. Take the cumulative sum over the entire array d , that is
$$d[i] = \sum_{j=0}^i d[j]$$
5. Add d to A

	0	1	2	3	4	5	6	7	8
D:	0	0	0	0	0	0	0	0	0



Mark Q₁

	0	1	2	3	4	5	6	7	8
D:	0	+1	0	0	-1	0	0	0	0



Cumulative Sum

	0	1	2	3	4	5	6	7	8
D:	0	1	1	1	0	0	0	0	0

One-dimensional difference arrays

This concept can be applied to multiple queries.

- Q1: Update elements at indices in $\{1, 2, 3\}$ by $+1$
- Q2: Update elements at indices in $\{2, 3\}$ by $+2$
- Q3: Update elements at indices in $\{5, 6, 7, 8\}$ by -1

Then form $d = d_1 + d_2 + d_3$.

	0	1	2	3	4	5	6	7	8
d_1 :	0	+1	0	0	-1	0	0	0	0
	0	1	2	3	4	5	6	7	8
d_2 :	0	0	+2	0	-2	0	0	0	0
	0	1	2	3	4	5	6	7	8
d_3 :	0	0	0	0	0	-1	0	0	0
						+			
	0	1	2	3	4	5	6	7	8
d :	0	+1	+2	0	-3	-1	0	0	0

One-dimensional difference arrays

The cumulative sum distributes over

$$d = d_1 + d_2 + d_3:$$

$$\begin{aligned} \sum_{j=0}^i d[j] &= \sum_{j=0}^i (d_1[j] + d_2[j] + d_3[j]) \\ &= \sum_{j=0}^i d_1[j] + \sum_{j=0}^i d_2[j] + \sum_{j=0}^i d_3[j] \end{aligned}$$

Hence, $\Sigma d = \Sigma d_1 + \Sigma d_2 + \Sigma d_3$.

Finally add Σd to A .

	0	1	2	3	4	5	6	7	8
$\Sigma d_1:$	0	1	1	1	0	0	0	0	0
	0	1	2	3	4	5	6	7	8
$\Sigma d_2:$	0	0	2	2	0	0	0	0	0
	0	1	2	3	4	5	6	7	8
$\Sigma d_3:$	0	0	0	0	0	-1	-1	-1	-1
						+			
	0	1	2	3	4	5	6	7	8
$\Sigma d:$	0	1	3	3	0	-1	-1	-1	-1

One-dimensional difference arrays

Non-formal difference array algorithm:

Input: Array A of size N and L queries of size at most N

1. Initialize array d of size N
2. For every query add two markers to d
3. Form the cumulative sum Σd
4. Add Σd to A

Runtime analysis:

- Initializing d in $\mathcal{O}(N)$
- Per query placing two markers, i.e. in total $\mathcal{O}(L)$ markers
- Forming Σd in $\mathcal{O}(N)$ by computing

$$\sum_{j=0}^i d[j] = d[i] + \sum_{j=0}^{i-1} d[j]$$

($N - 2$ additions, $N - 1$ reads)

- Adding Σd to A in $\mathcal{O}(N)$.

In total: $\mathcal{O}(N + L)$.

Two-dimensional difference arrays

This concept can easily be generalized to 2D arrays.

*	*	*	*	*	*	*	*
*	$+C$	$+C$	$+C$	$+C$	$+C$	*	*
*	$+C$	$+C$	$+C$	$+C$	$+C$	*	*
*	$+C$	$+C$	$+C$	$+C$	$+C$	*	*
*	*	*	*	*	*	*	*

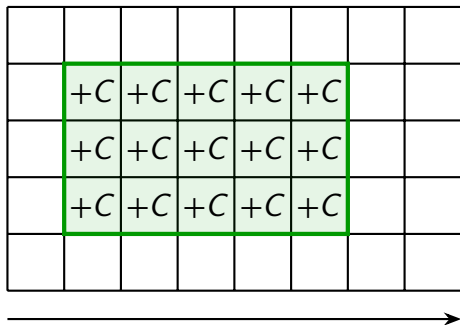
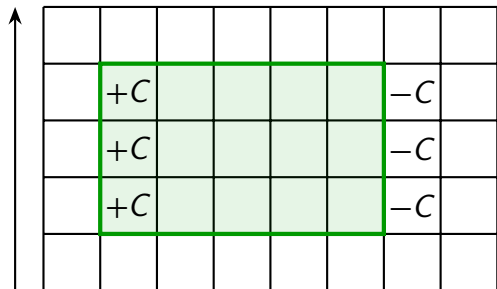
Naively applying the constant update to a 5×8 array A.

	$-C$					$+C$	

Placing *four* markers in the difference array.

Two-dimensional difference arrays

Taking cumulative sums in both directions yields the correct updates.



d -dimensional difference arrays?

- This heuristic can easily be extended to d -dimensional arrays by placing 2^d markers for every query.
- However for dimensions $d \geq 4$ this is hard to conceptually verify.
- Furthermore, "naive" proofs without any additional tools are bound to get very messy.
- This is where *Discrete Calculus* comes into play.
- To leverage these tools, we first need to formally describe our situation.

The formal framework

We view arrays as maps over *finite grid domains*.

Definition: A finite subset $D \subseteq \mathbb{Z}^d$ is called *finite grid domain* if there exist $n_1, \dots, n_d \in \mathbb{Z}$ and $m_1, \dots, m_d \in \mathbb{Z}$ with $n_i \leq m_i$ for all $i \in \{1, \dots, d\}$ such that $D = \prod_{i=1}^d \llbracket n_i, m_i \rrbracket$. A *discrete map* is a map $f : D \rightarrow \mathbb{R}$ over a finite grid domain $D \subseteq \mathbb{Z}^d$.

A *d-dimensional array* is a discrete function $f : D \rightarrow \mathbb{R}$ for which the finite grid domain D is of the form $D = \prod_{i=1}^d \llbracket 0, m_i \rrbracket \subseteq \mathbb{Z}^d$ where $m_i \in \mathbb{N}$ for all $1 \leq i \leq d$.

The notation $\llbracket a, b \rrbracket$ for $a \leq b \in \mathbb{Z}$ means the *discrete interval*

$$\llbracket a, b \rrbracket := \{m \in \mathbb{Z} \mid a \leq m \leq b\}.$$

Definition: Let $D = \prod_{i=1}^d \llbracket n_i, m_i \rrbracket \subseteq \mathbb{Z}^d$ be a finite grid domain. A *constant update query* q on D is a discrete function $q : D \rightarrow \mathbb{R}$ such that there exists a subset $D_q = \prod_{i=1}^d \llbracket a_i, b_i \rrbracket \subseteq D$ called the *update range* of q and a constant $C \in \mathbb{R}$ called the *update value* of q with

$$q(k_1, \dots, k_d) := \begin{cases} C, & \text{if } (k_1, \dots, k_d) \in D_q, \\ 0, & \text{else} \end{cases}$$

for $(k_1, \dots, k_d) \in D$, that is $q = C \cdot \mathbb{1}_{D_q}$. The *size* of a constant update query $q : D \rightarrow \mathbb{R}$ is the cardinality of the update range D_q .

Let $f : D \rightarrow \mathbb{R}$ with $D = \prod_{i=1}^d \llbracket n_i, m_i \rrbracket$ be a discrete function and $q_1, \dots, q_L$ a series of constant update queries.

Applying the series of queries to f means forming the updated discrete function $F : D \rightarrow \mathbb{R}$ as

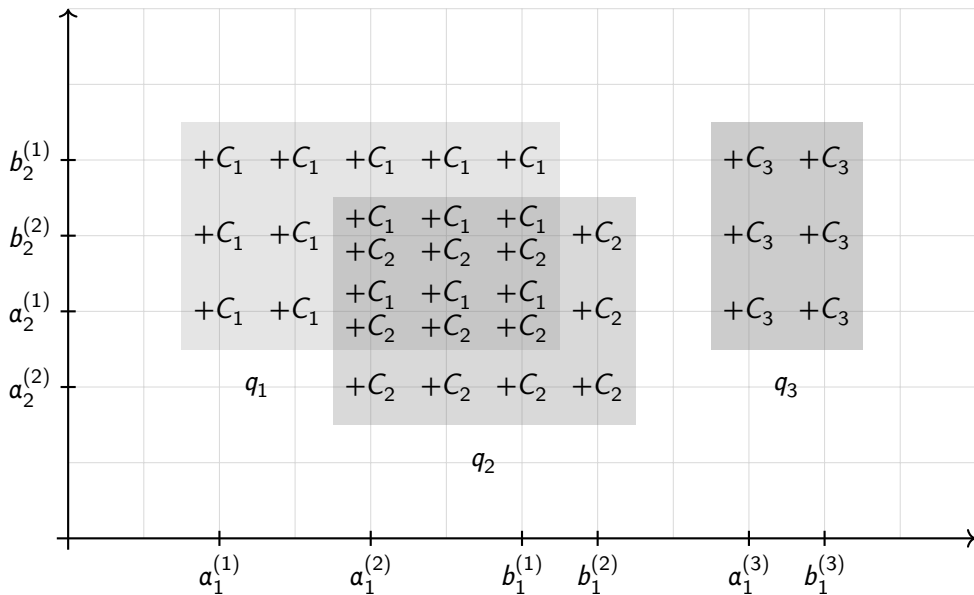
$$F := f + \sum_{i=1}^L q_i.$$

A naive algorithm

Input: A discrete map $f : D \rightarrow \mathbb{R}$ with $D = \prod_{i=1}^d \llbracket n_i, m_i \rrbracket$ and a series of constant update queries $q_1, \dots, q_L$ on f over the update ranges $D_j = \prod_{i=1}^d \llbracket a_i^{(j)}, b_i^{(j)} \rrbracket$ with update values C_j for $1 \leq j \leq L$

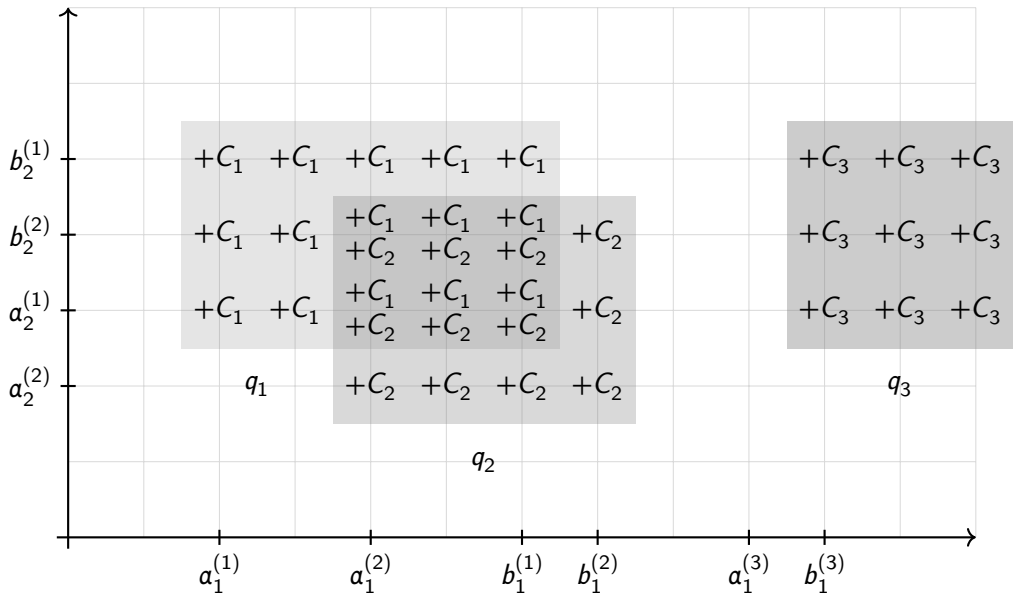
Output: The updated function $F = f + \sum_{i=1}^L q_i$

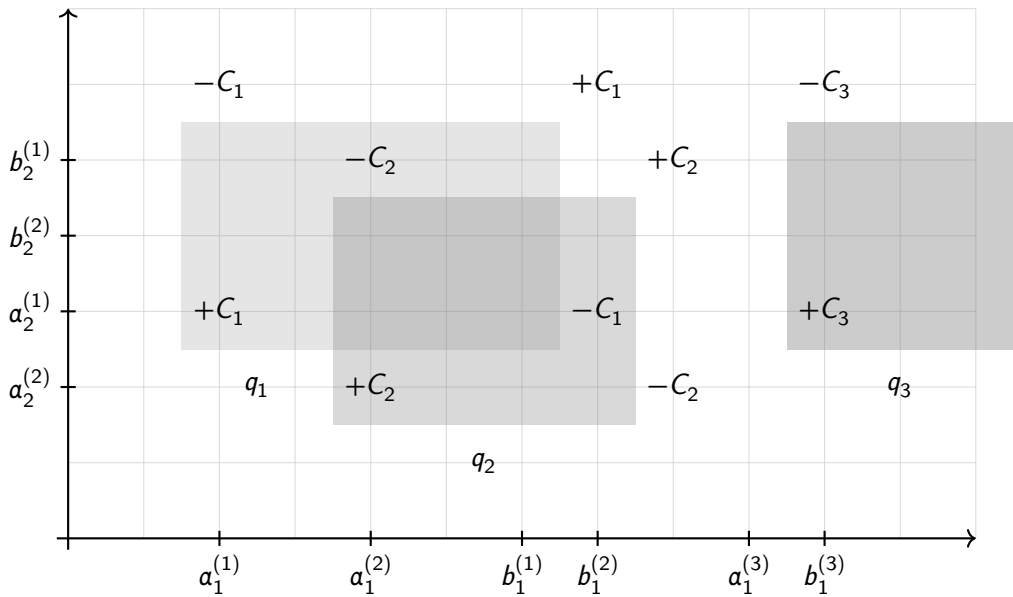
```
1:  $F \leftarrow$  The map  $f : D \rightarrow \mathbb{R}$        $\triangleright$  For an in situ implementation set  $F = f$  without any copy
   operations
2: for  $j \in \llbracket 1, L \rrbracket$  do
3:   for  $k \in D_j$  do
4:      $F(k) \leftarrow F(k) + C_j$ 
5:   end for
6: end for
   return  $F$ 
```

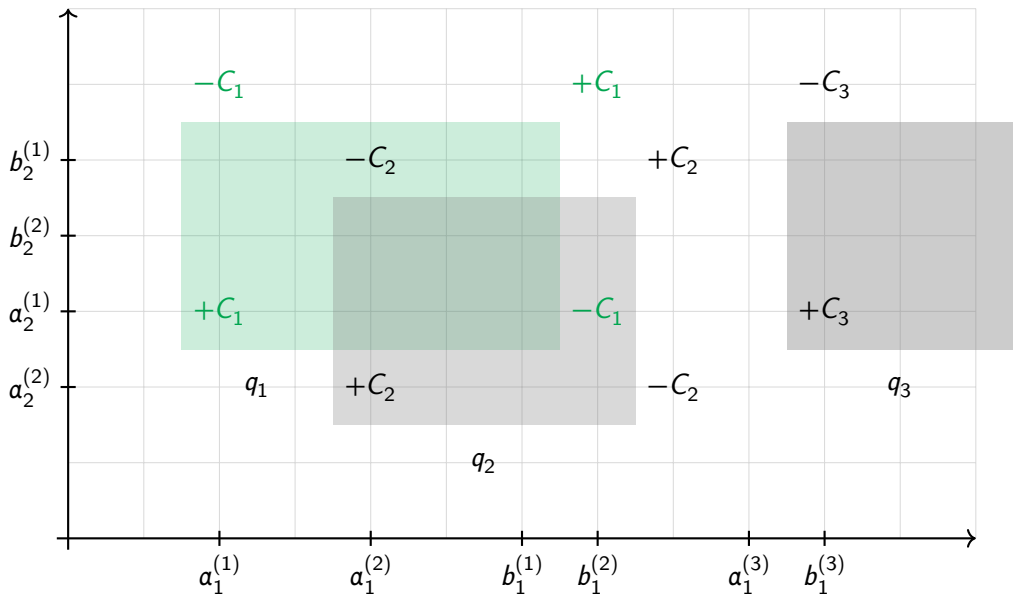


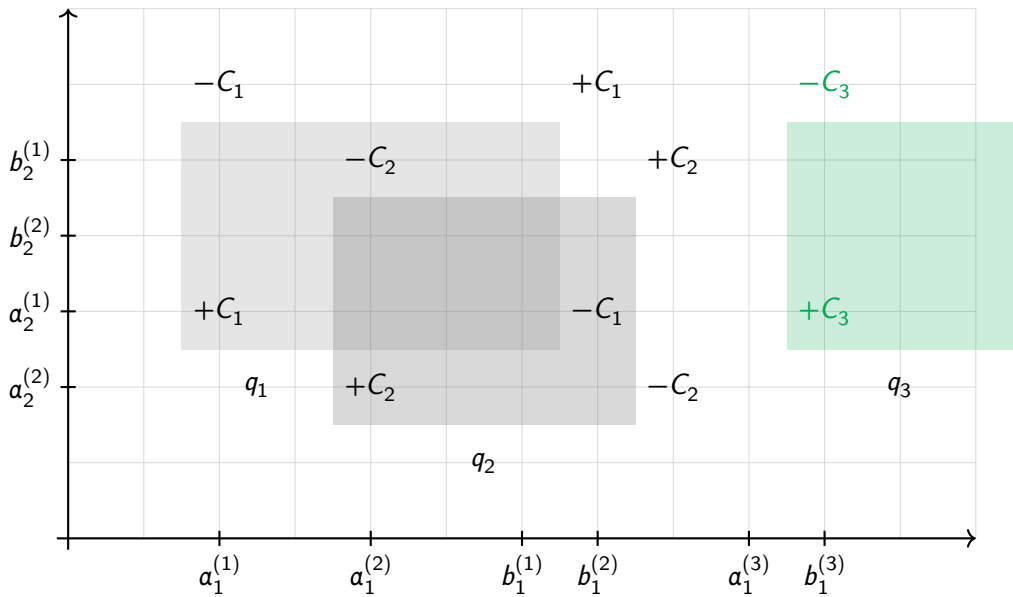
Proposition: Let $f : D \subseteq \mathbb{Z}^d \rightarrow \mathbb{R}$ be a discrete function with $D = \prod_{i=1}^d \llbracket n_i, m_i \rrbracket$ and $N = \text{card}(D)$, and let $q_1, \dots, q_Q$ be a series of L constant update queries on f each of size at most M .

- Assuming that for each $k \in D$ the function value $f(k)$ can be obtained in constant time, the naive algorithm computes the updated function $F = f + \sum_{i=1}^Q q_i$ in $\mathcal{O}(N + LM)$.
- If the algorithm can be implemented in situ, its runtime lies in $\mathcal{O}(LM)$.









Definition: Let $D = \prod_{i=1}^d \llbracket n_i, m_i \rrbracket \subseteq \mathbb{Z}^d$ be a finite grid domain and $q_1, \dots, q_L$ a series of constant update queries on D over the update ranges $D_j = \prod_{i=1}^d \llbracket a_i^{(j)}, b_i^{(j)} \rrbracket$ with update values C_j for $j \in \{1, \dots, L\}$.

The *difference array* $\Delta[q_1, \dots, q_L] : D \rightarrow \mathbb{R}$ is defined as

$$\Delta[q_1, \dots, q_L] = \sum_{j=1}^L \sum_{p \in M_j} (-1)^{N_j(p)} \cdot C_j \cdot \delta_p,$$

where $M_j = \prod_{i=1}^d \left(\{a_i^{(j)}\} \cup \{b_i^{(j)} + 1 \mid b_i^{(j)} \neq m_i\} \right)$ and $N_j(p) = \text{card}\{p_i \mid p_i = b_i^{(j)} + 1\}$ for $j \in \{1, \dots, L\}$.

The notation δ_p means the Dirac-delta given by

$$\delta_p(k) = \begin{cases} 1, & \text{if } k = p, \\ 0, & \text{else.} \end{cases}$$

Procedure: DIFFARR($D, q_1, \dots, q_Q$)

Input: A finite grid domain $D = \prod_{i=1}^d \llbracket n_i, m_i \rrbracket \subseteq \mathbb{Z}^d$ and a series of constant update queries $q_1, \dots, q_Q$ on D over the update ranges $D_j = \prod_{i=1}^d \llbracket a_i^{(j)}, b_i^{(j)} \rrbracket$ with update values C_j for $1 \leq j \leq Q$

Output: The difference array $\Delta[q_1, \dots, q_Q]$

```
1:  $d \leftarrow$  Zero map  $d : D \rightarrow \mathbb{R}$ 
2: for  $j \in \llbracket 1, Q \rrbracket$  do
3:    $M_j \leftarrow \prod_{i=1}^d \left( \{a_i^{(j)}\} \cup \{b_i^{(j)} + 1 \mid b_i^{(j)} \neq m_i\} \right)$ 
4:   for  $p \in M_j$  do
5:      $N(p) \leftarrow \text{card}\{p_i \mid p_i = b_i^{(j)} + 1\}$ 
6:      $d(p) \leftarrow d(p) + (-1)^{N(p)} \cdot C_j$ 
7:   end for
8: end for
9: return  $d$ 
```

Proposition: Let $D = \prod_{i=1}^d \llbracket n_i, m_i \rrbracket \subseteq \mathbb{Z}^d$ be a finite grid domain, $N = \text{card}(D)$ and let $q_1, \dots, q_L$ be a series of constant update queries on D of arbitrary size. The difference array algorithm computes the difference array $\Delta[q_1, \dots, q_Q]$ in $\mathcal{O}(N + L)$.

How does this formal notion of a difference array help us in computing the updated map $F = f + \sum_{i=0}^L q_i$ more efficiently?

From the 1D/2D heuristic: Cumulatively summing over $\Delta[q_1, \dots, q_L]$ should yield the total update $\sum_{i=0}^L q_i$.

Roadmap: How does Discrete Calculus come into play?

- We will see that a difference array $\Delta[q_1, \dots, q_L]$ is the sum of discrete derivatives of the queries $q_1, \dots, q_L$, that is

$$\Delta[q_1, \dots, q_L] = \sum_{i=0}^L \Delta q_i.$$

- Cumulatively summing over $\Delta[q_1, \dots, q_L]$ will be the discrete analogon of integration and thus will be the inverse operation to discrete differentiation, that is

$$\Sigma(\Delta[q_1, \dots, q_L]) = \sum_{i=0}^L \Sigma(\Delta q_i) = \sum_{i=0}^L q_i.$$

Foundations of Discrete Calculus

Definition: The i -th backward difference Δ_i is the linear operator assigning to every discrete function $f : D \subseteq \mathbb{Z}^d \rightarrow \mathbb{R}$ with $D = \prod_{i=1}^d \llbracket n_i, m_i \rrbracket \subseteq \mathbb{Z}^d$ the function $\Delta_i f : D \rightarrow \mathbb{R}$ given by

$$\begin{aligned}\Delta_i f(k_1, \dots, k_d) &:= f(k_1, \dots, k_{i-1}, k_i, k_{i+1}, \dots, k_d) \\ &\quad - f(k_1, \dots, k_{i-1}, k_i - 1, k_{i+1}, \dots, k_d)\end{aligned}$$

for all $(k_1, \dots, k_d) \in D$ with $k_i > n_i$ and

$$\Delta_i f(k_1, \dots, k_d) := f(k_1, \dots, k_{i-1}, k_i, k_{i+1}, \dots, k_d)$$

for all $(k_1, \dots, k_d) \in D$ with $k_i = n_i$.

Definition (continued): For shorthand notation, we use

$$\Delta := \Delta_1 \circ \dots \circ \Delta_d$$

to denote the *mixed backward difference* Δ .

Definition: The i -th (inclusive) prefix sum Σ_i is the linear operator assigning to every discrete function $f : D \subseteq \mathbb{Z}^d \rightarrow \mathbb{R}$ with $D = \prod_{j=1}^d \llbracket n_j, m_j \rrbracket$ the function $\Sigma_i f : D \rightarrow \mathbb{R}$ given by

$$\Sigma_i f(k_1, \dots, k_d) := \sum_{t=n_i}^{k_i} f(k_1, \dots, k_{i-1}, t, k_{i+1}, \dots, k_d)$$

for all $(k_1, \dots, k_d) \in D$. Again for shorthand notation, we use

$$\Sigma := \Sigma_1 \circ \dots \circ \Sigma_d$$

to denote the (inclusive) prefix sum Σ .

It is not hard to show that backward differences and prefix sums commute, that is for every i and j we have $\Sigma_i \circ \Delta_j = \Delta_j \circ \Sigma_i$.

In particular, $\Sigma \circ \Delta = (\Sigma_1 \circ \Delta_1) \circ \dots \circ (\Sigma_d \circ \Delta_d) = \Delta \circ \Sigma$.

Proposition: The i -th backward difference Δ_i and the i -th prefix sum Σ_i are inverse to each other for all i . That is, for every discrete function $f : D \rightarrow \mathbb{R}$ with $D = \prod_{i=1}^d \llbracket n_i, m_i \rrbracket \subseteq \mathbb{Z}^d$ we have

$$\Delta_i \circ \Sigma_i(f) = f \quad \text{and} \quad \Sigma_i \circ \Delta_i(f) = f$$

for all $i \in \{1, \dots, d\}$. In particular, the mixed backward difference $\Delta = \Delta_1 \circ \dots \circ \Delta_d$ and the prefix sum $\Sigma = \Sigma_1 \circ \dots \circ \Sigma_d$ are inverse to each other.

Another important property of prefix sums is that they can be computed efficiently.

Procedure: PREFIXSUM

Input: A discrete map $f : D \rightarrow \mathbb{R}$ with $D = \prod_{i=1}^d \llbracket n_i, m_i \rrbracket \subseteq \mathbb{Z}^d$

Output: The prefix sum $\Sigma(f) : D \rightarrow \mathbb{R}$

```
1:  $S \leftarrow$  The discrete map  $f : D \rightarrow \mathbb{R}$ 
2: for  $j \in \llbracket 1, d \rrbracket$  do
3:   for  $(k_1, \dots, k_{j-1}, k_{j+1}, \dots, k_d) \in \prod_{\substack{i=1 \\ i \neq j}}^d \llbracket n_i, m_i \rrbracket$  do
4:     for  $t \in \llbracket n_j + 1, m_j \rrbracket$  do
5:        $S(k_1, \dots, k_{j-1}, t, k_{j+1}, \dots, k_d) \leftarrow S(k_1, \dots, k_{j-1}, t - 1, k_{j+1}, \dots, k_d)$ 
6:          $\quad + S(k_1, \dots, k_{j-1}, t, k_{j+1}, \dots, k_d)$ 
7:     end for
8:   end for
9: return  $S$ 
```

Proposition: Let $f : D \rightarrow \mathbb{R}$ be a discrete map with $D = \prod_{i=1}^d \llbracket n_i, m_i \rrbracket \subseteq \mathbb{Z}^d$. The PREFIXSUM algorithm computes the prefix sum $\Sigma f : D \rightarrow \mathbb{R}$ in $\mathcal{O}(N)$ with $N = \text{card}(D)$.

Indicator functions and Heaviside step functions

Definition: Let $p = (p_1, \dots, p_d) \in \mathbb{Z}^d$. The *Heaviside step function* $H_p : \mathbb{Z}^d \rightarrow \mathbb{R}$ is the indicator function given by

$$H_p(k_1, \dots, k_d) := \begin{cases} 1, & \text{if } \forall 1 \leq i \leq d : k_i \geq p_i, \\ 0, & \text{else} \end{cases}$$

for $(k_1, \dots, k_d) \in \mathbb{Z}^d$.

Heaviside step functions are specific indicator functions with very simple mixed backward differences.

Proposition: Let $D = \prod_{i=1}^d \llbracket n_i, m_i \rrbracket \subseteq \mathbb{Z}^d$ be a finite grid domain and $p = (p_1, \dots, p_d) \in D$. The mixed backward difference ΔH_p of the Heaviside step function $H_p : D \rightarrow \mathbb{R}$ is given by

$$\Delta H_p(k_1, \dots, k_d) = \delta_p := \begin{cases} 1, & \text{if } \forall 1 \leq i \leq d : k_i = p_i, \\ 0, & \text{else.} \end{cases}$$

Proof. ■ It is straight forward to see that $H_p = \prod_{i=1}^d H_{p_i}$

- One can show that for such products we have $\Delta H_p = \prod_{i=1}^d \Delta H_{p_i}$
- Show $\Delta H_{p_i} = \delta_{p_i}$ by a simple case distinction
- Finally, $\Delta H_p = \prod_{i=1}^d \delta_{p_i} = \delta_{(p_1, \dots, p_d)}$

□

Proposition: Let $D = \prod_{i=1}^d \llbracket a_i, b_i \rrbracket \subseteq \mathbb{Z}^d$ be a finite grid domain. Then, the indicator function $\mathbb{1}_D : \mathbb{Z}^d \rightarrow \mathbb{R}$ can be written as

$$\mathbb{1}_D = \sum_{p \in \prod_{i=1}^d \{a_i, b_i+1\}} (-1)^{N(p)} H_p,$$

where $N(p) = \text{card}\{p_i \mid p_i = b_i + 1\}$.

Proof. ■ It is not hard to see that $\mathbb{1}_D(k_1, \dots, k_d) = \prod_{i=1}^d \mathbb{1}_{\llbracket a_i, b_i \rrbracket}(k_i)$

- By a simple case distinction we see that $\mathbb{1}_{\llbracket a_i, b_i \rrbracket}(k_i) = H_{a_i}(k_i) - H_{b_i+1}(k_i)$
- Hence, $\mathbb{1}_D(k_1, \dots, k_d) = \prod_{i=1}^d (H_{a_i}(k_i) - H_{b_i+1}(k_i))$
- Using the general distribution formula for products yields

$$\prod_{i=1}^d (H_{a_i}(k_i) - H_{b_i+1}(k_i)) = \sum_{J \subseteq I} (-1)^{\text{card}(J)} \left(\prod_{i \in I \setminus J} H_{a_i}(k_i) \right) \left(\prod_{j \in J} H_{b_j+1}(k_j) \right),$$

where $I = \{1, \dots, d\}$.

Corollary: Let $D = \prod_{i=1}^d \llbracket n_i, m_i \rrbracket \subseteq \mathbb{Z}^d$ be a finite grid domain and let $q : D \rightarrow \mathbb{R}$ be a constant update query on D with update range $D_q = \prod_{i=1}^d \llbracket a_i, b_i \rrbracket$ and update value $C \in \mathbb{R}$. Then,

$$q = \sum_{p \in M} (-1)^{N(p)} \cdot C \cdot H_p,$$

where $M = \prod_{i=1}^d (\{a_i\} \cup \{b_i + 1 \mid b_i \neq m_i\})$ and $N(p) = \text{card}\{p_i \mid p_i = b_i + 1\}$. In particular, the mixed backward difference Δq of q can be expressed as

$$\Delta q = \sum_{p \in M} (-1)^{N(p)} \cdot C \cdot \delta_p.$$

Proof. ■ By definition $q = C \cdot \mathbb{1}_{D_q}$

- Writing the indicator function by Heaviside step functions yields the first claim
- The second claim follows by the linearity of Δ and as previously shown $\Delta H_p = \delta_p$ □

Recall the formal definition of difference arrays:

Definition: Let $D = \prod_{i=1}^d \llbracket n_i, m_i \rrbracket \subseteq \mathbb{Z}^d$ be a finite grid domain and $q_1, \dots, q_L$ a series of constant update queries on D . The *difference array* $\Delta[q_1, \dots, q_Q] : D \rightarrow \mathbb{R}$ is defined as

$$\Delta[q_1, \dots, q_L] = \sum_{j=1}^L \sum_{p \in M_j} (-1)^{N_j(p)} \cdot C_j \cdot \delta_p,$$

where $M_j = \prod_{i=1}^d \left(\{a_i^{(j)}\} \cup \{b_i^{(j)} + 1 \mid b_i^{(j)} \neq m_i\} \right)$ and $N_j(p) = \text{card}\{p_i \mid p_i = b_i^{(j)} + 1\}$ for $1 \leq j \leq Q$.

- With the corollary on the last slide we see that

$$\Delta q_j = \sum_{p \in M_j} (-1)^{N_j(p)} \cdot C_j \cdot \delta_p$$

for all $j \in \{1, \dots, L\}$.

- Hence, we indeed have

$$\Delta[q_1, \dots, q_L] = \sum_{j=1}^L \Delta q_j.$$

- The prefix sum and the mixed backwards difference are inverse to each other, thus

$$\Sigma(\Delta[q_1, \dots, q_L]) = \sum_{j=1}^L \Sigma(\Delta q_j) = \sum_{j=1}^L q_j.$$

- We have seen that $\Delta[q_1, \dots, q_L]$ can be computed in $\mathcal{O}(N + L)$.
- We have seen that the prefix sum $\Sigma(\Delta[q_1, \dots, q_L])$ can be computed in $\mathcal{O}(N)$.
- The addition $f + \Sigma(\Delta[q_1, \dots, q_L])$ has time complexity $\mathcal{O}(N)$.
- In total: Computing $f + \Sigma(\Delta[q_1, \dots, q_L])$ can be done in $\mathcal{O}(N + L)$.

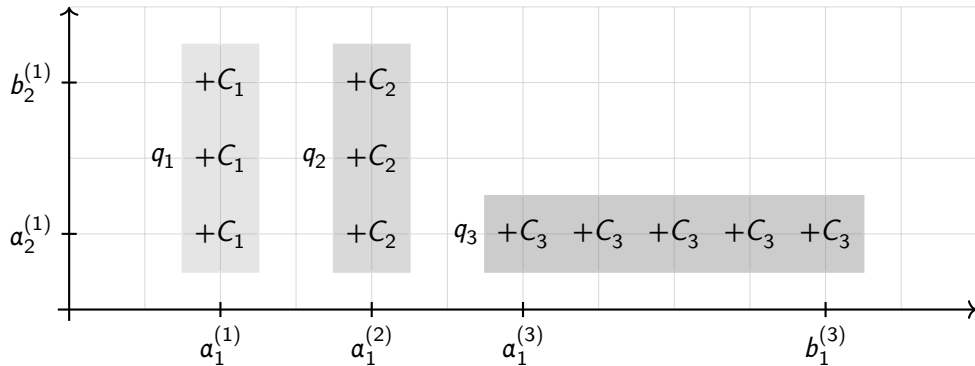
Input: A discrete map $f : D \rightarrow \mathbb{R}$ with $D = \prod_{i=1}^d \llbracket n_i, m_i \rrbracket$ and a series of constant update queries $q_1, \dots, q_Q$ on D

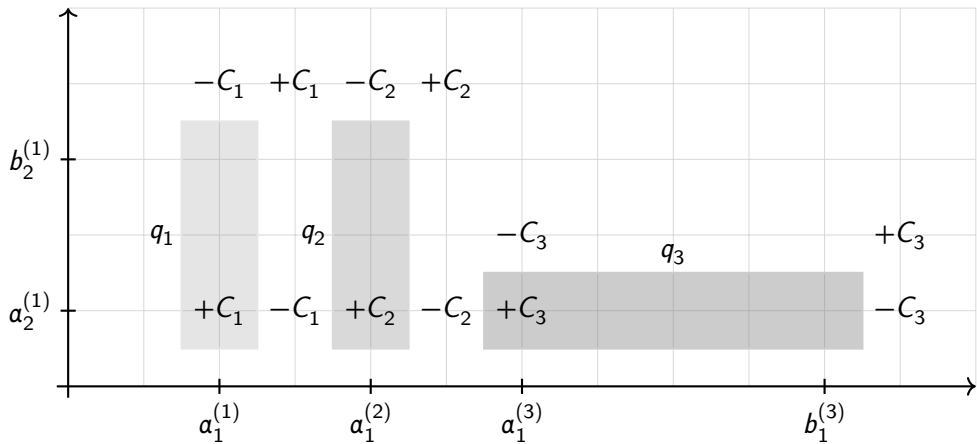
Output: The updated map $F = f + \sum_{i=1}^Q q_i$

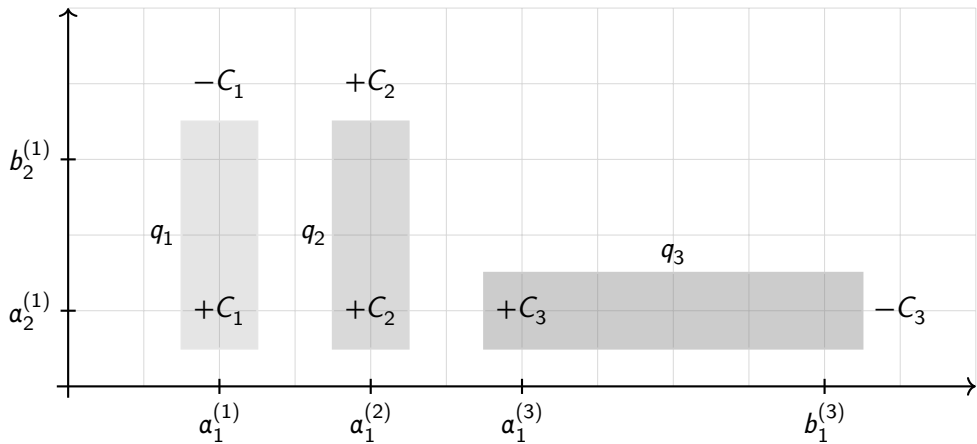
- 1: $d \leftarrow \text{DIFFARR}(D, q_1, \dots, q_Q)$
- 2: $\Sigma d \leftarrow \text{PREFIXSUM}(d)$
- 3: $F \leftarrow f + \Sigma d$
- 4: **return** F

Proposition: Let $f : D \subseteq \mathbb{Z}^d \rightarrow \mathbb{R}$ be a discrete function with $D = \prod_{i=1}^d \llbracket n_i, m_i \rrbracket$ and $N = \text{card}(D)$, and let $q_1, \dots, q_L$ be a series of constant update queries on D of arbitrary size. Assuming that for each $k \in D$ the function value $f(k)$ can be obtained in constant time, the algorithm above computes the updated map $F = f + \sum_{i=1}^L q_i$ in $\mathcal{O}(N + L)$.

Flat update queries







Consecutive update queries

